

CPA Site Delivery Pipeline

Automated Website Build & Publishing Architecture

Next.js · GitHub · Vercel · Claude Design · No CMS Required

Prepared for technical review — May 2026

OVERVIEW

What We're Building

CountingFive currently builds CPA firm websites on WordPress — a platform that requires manual content entry, theme management, plugin maintenance, and per-site configuration. As we move toward a fully automated onboarding pipeline (intake → content generation → site delivery), WordPress becomes the bottleneck.

This document outlines a replacement architecture with three components: **Next.js** (the site framework), **GitHub** (content storage and version control), and **Vercel** (hosting and deployment). Since CountingFive handles all content updates — clients never log in to edit — there is no need for a CMS layer. Content lives as files in the repo, managed entirely by our team.

Alongside the content pipeline, a separate **Claude Design workflow** handles per-client visual styling. A human designer works with Claude to style the site's 12–15 component blocks — adjusting CSS custom properties to give each client a distinct look — before the final site is published. Content and design are developed independently and both must be complete before anything goes live.

The Core Idea — Three Lanes, One Publish

The content pipeline generates all copy, SEO metadata, and structured assets automatically. In parallel, a human-led Claude Design session styles the 12–15 site blocks for that client. When both are approved, a single 'Publish to Site' action commits everything to GitHub and Vercel auto-deploys. The result is a live, uniquely styled website with no manual content entry and no CMS login required.

THE STACK

Technology Choices

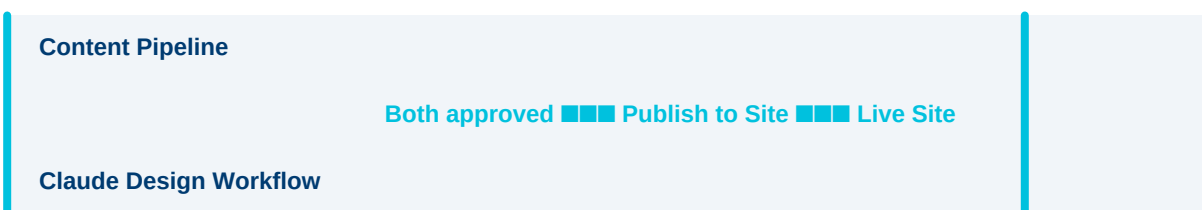
Layer	Tool	Role
Site Framework	Next.js 15	React-based, statically generated — same stack as our onboarding app
Content Storage	MDX files in GitHub	Each page is a .mdx file with frontmatter (meta, slug, palette). Version-controlled, diff-able, editable via Cursor or Claude Code
Hosting	Vercel	Auto-deploys on every GitHub push — handles SSL, CDN, and custom domains
Source Control	GitHub	One private repo per client site, branched from the CPA template

Design System	Tailwind + CSS vars	12–15 component blocks built with named CSS custom properties. Per-client theme.css overrides those variables — no structural changes needed per client
Claude Design	Claude Project	Dedicated Claude Project loaded with the block library and design context. Human designer works with Claude per client to produce a theme.css file from the client's brand data
Updates	Pipeline or Cursor/Claude Code	Structured updates re-run the pipeline and push new files. Quick edits go direct via Cursor, Claude Code, or the GitHub web editor

WORKFLOW

How It Works — End to End

The workflow has three lanes: a one-time template build, a per-client content pipeline, and a per-client Claude Design session. Content and design run independently — both must be complete before the site is published.



Phase A — One-Time Template Build

- 1 Build the CPA Firm Next.js Template**

Create a polished Next.js 15 site with page layout variants (homepage, service page, team, about, contact, blog), the CountingFive design system baked in (navy/cyan palette, Inter/Open Sans fonts), and MDX as the content source. Built once — reused for every client.
- 2 Build 12–15 Component Blocks with CSS Custom Properties**

Each block (hero, service cards, team section, testimonials, CTA strip, footer, etc.) exposes a set of named CSS custom properties: `--hero-bg`, `--card-radius`, `--section-spacing`, `--accent-weight`, and so on. These are the handles the Claude Design workflow adjusts per client. No structural changes are ever needed — only the values change.
- 3 Set Up the Claude Design Project**

Create a dedicated Claude Project loaded with the block library, the CSS variable reference, design guidelines for CPA firm aesthetics, and example before/after theme files. This project is reused for every client's design session — it builds up context and examples over time.
- 4 Define the MDX Content Schema**

Establish the frontmatter structure that every page file uses: slug, page type, H1, meta title, meta description, sections (heading + body + optional CTA), FAQ block, JSON-LD, and palette overrides. This is a simple TypeScript type, not a third-party service.
- 5 Build the Pipeline → GitHub Push Step**

Add a 'Publish to Site' button to the admin content hub. When clicked, it commits both the approved MDX content files and the client's theme.css to the client's GitHub repo via the GitHub API. Vercel auto-deploys on the commit.
- 6 GitHub Template Repo + Vercel Config**

Set up a private GitHub template repository. Each new client gets their own repo forked from the template. Vercel connects to the repo and auto-deploys on every push.

Phase B — Claude Design Workflow (Per-Client, Human-Led)

This workflow runs in parallel with the content pipeline — it can start as soon as the palette and brand data are available. A human designer drives the session; Claude handles the CSS mechanics.

- 1 Open the Claude Design Project**

Start a new session in the dedicated Claude Design Project. The project already has the full block library, CSS variable reference, and design guidelines loaded as context.
- 2 Feed in the Client's Brand Data**

Paste the client's palette (6 swatches + tone signal), brand voice excerpt, niche focus, firm name, team size, and service count. This is a direct export from the content pipeline's schema_data — no manual data entry.
- 3 Claude Proposes a Starting Theme**

Claude generates an initial theme.css with values set for all 12–15 blocks, reasoning from the brand data. For example: a conservative estate planning firm gets tighter spacing, a smaller border radius, and restrained accent color use. A modern advisory firm gets bolder typography and more open layouts.
- 4 Human Reviews and Refines**

The designer reviews Claude's proposals block by block — reacting, redirecting, approving. This is the creative judgment layer: Claude handles consistency and CSS correctness, the human handles whether it actually feels right for this firm. The conversation stays in the Project and builds up useful context for future clients.
- 5 Output: theme.css Approved**

The session produces a single theme.css file containing all CSS custom property overrides for this client. It's clean, scoped, and ready to drop into the client's repo.

Phase C — Per-Client Site Delivery

Once both the content pipeline and the Claude Design workflow are complete, the site is ready to publish. Steps 3 and 4 are fully automated.

- 1 Onboarding Chat (existing)**

Client completes the AI-guided intake conversation. Firm info, team bios, services, niches, and brand voice are collected and stored in the session schema.
- 2 Content Pipeline Runs (existing)**

Admin triggers the 6-phase content workflow: palette → sitemap → research → outline review → content generation → deliverables. Runs independently of the design workflow.
- 3 Admin Clicks 'Publish to Site'**

Once both content and theme.css are approved, a single publish action commits the MDX content files and the client's theme.css to the GitHub repo via the GitHub API. Both land in the same commit — one deploy covers everything.

4**Vercel Auto-Deploys (automatic)**

Vercel detects the commit and rebuilds the site with the new content and styling applied. No further intervention required. The site is live.

5**Domain & DNS Setup**

Point the client's domain to Vercel, add the custom domain in the project settings, and verify SSL provisioning. This is the only manual step per client.

6**QA & Handoff**

Review the live site — content, styling, and mobile layout. Future content updates go through the pipeline; visual updates go back through the Claude Design Project.

BUILD REQUIREMENTS

What Needs to Be Built

All required development work is listed below. Everything is one-time — none of it repeats per client.

Template Development (One-Time)

Next.js CPA Firm Template + 12–15 Component Blocks

Full Next.js 15 site: App Router, Tailwind design system, MDX page rendering, 12–15 component blocks (hero, service cards, team section, testimonials, CTA strip, stats bar, FAQ, blog index/post, header, footer, and others). Each block built with named CSS custom properties as styling handles. Mobile responsive, Core Web Vitals optimized.

MDX Content Schema & Type Definitions

Define the TypeScript types and Zod validation for page frontmatter. Ensures every MDX file produced by the pipeline matches what the Next.js template expects. Simple and self-contained — no third-party service required.

Pipeline → GitHub Push Step

New 'Publish to Site' step in the admin content hub. Reads generated_pages from the database, serializes each page as a .mdx file with correct frontmatter, and commits the batch to the client's GitHub repo via the GitHub API. Includes per-client repo config stored in the sessions table, error handling, and publish status tracking.

GitHub Template Repo + Per-Client Setup

Create the template repo with a clear per-client config file (brand colors, logo path, contact info, Google Analytics ID). Document the setup checklist: fork repo, set Vercel env vars, connect domain.

CSS Custom Property System

Define the full set of CSS custom properties that the 12–15 blocks expose as styling handles: spacing scale, border radius, shadow depth, typography weights, accent color application rules, and background variants. This is the interface between the template and the Claude Design workflow — every per-client theme.css writes to these variables and nothing else.

Claude Design Project Setup

Create and configure the dedicated Claude Project for the design workflow. Load the block library documentation, CSS variable reference, CPA firm design guidelines, and a set of example theme files as project context. Write the system prompt that orients Claude as a CPA firm web designer. This project is reused for every client — it compounds value as sessions accumulate.

Pipeline → GitHub Push Step (Content + Theme)

New 'Publish to Site' step in the admin content hub. Reads generated _pages from the database, serializes each page as a .mdx file, bundles the approved theme.css, and commits everything to the client's GitHub repo in a single push. Includes publish-status tracking in the admin.

GitHub Template Repo + Per-Client Setup

Create the template repo with a per-client config file (firm name, logo path, contact info, GA ID). Document the setup checklist: fork repo, set Vercel env vars, connect domain.

One-Time Build — Key Points

All items above are fixed, one-time work. Once complete, none of it repeats for any subsequent client.

Removing Sanity CMS from the architecture eliminated a significant portion of the original build scope and all per-client CMS account setup.

Per-Client Setup (After Template Is Built)

Each new client requires the following setup steps before publishing.

- Fork the template repo and set the brand config file (firm name, logo path, contact info)
- Connect the repo to Vercel and set environment variables
- Run the content pipeline through to deliverables approval
- Run the Claude Design session — export the approved theme.css
- Click 'Publish to Site' in the admin — commits content + theme.css together, Vercel auto-deploys
- Add the client's custom domain in Vercel and verify DNS
- QA review of the live site — content, styling, and mobile layout

COST ANALYSIS

Costs Per Site

Removing the CMS layer brings per-site infrastructure cost to effectively zero. All platform costs are flat monthly fees that cover the entire CountingFive client portfolio.

Monthly Infrastructure — CountingFive Level (All Sites)

Service	Plan	Cost/mo	Covers
Vercel	Pro Team	\$20	Team account — required for custom domains and advanced features
GitHub	Free / Team	\$0–4/user	Private repos for all client sites
Anthropic API	Pay per use	Existing	Already used in the content pipeline — no new cost

Per-Site Cost Breakdown (Recurring)

With no CMS subscription and no per-site plugin licenses, per-client recurring costs are limited to hosting and domain registration.

Item	Monthly	Notes
Vercel hosting	\$10	Per-project hosting on Vercel Pro
CMS / content service	\$0	No CMS — content is files in the GitHub repo
GitHub repo	\$0	Covered by team plan
SSL certificate	\$0	Auto-provisioned by Vercel
Domain registration	~\$1–1.50	Client-owned; ~\$12–18/year — already exists in most cases
TOTAL PER SITE	~\$11–11.50/mo	vs. \$30–50+/mo on WordPress

Compared to WordPress

WordPress typically requires: WP Engine or Kinsta hosting \$15–30/mo per site, premium theme \$50–200 one-time, essential plugin licenses \$100–300/year, maintenance 1–2 hrs/mo per site.

Across 20+ client sites, that's \$300–600+/month in hosting alone, plus ongoing labor. This architecture eliminates all of that.

How Content Updates Work Post-Launch

Since CountingFive owns all content updates, every change follows one of two paths:

Update Type	How It's Done
Full page rewrite or new page	Re-run the content pipeline for the affected pages, click 'Publish to Site'
Small copy edit (headline, paragraph, CTA)	Edit the .mdx file directly in Cursor or Claude Code, commit to GitHub
Team member or service update	Update the relevant MDX file and commit — or re-run the relevant pipeline section
New page added to sitemap	Add the page to the pipeline sitemap, run content generation, click 'Publish to Site'

ALTERNATIVES CONSIDERED

Why Not Webflow, Framer, or WordPress?

Three other platforms were evaluated. The table below summarizes the key trade-offs.

Criteria	Next.js + GitHub	Webflow	Framer	WordPress
Programmatic content push	Full GitHub API	Limited	Not designed for it	Via REST API only
Per-site cost	~\$11–12/mo	\$23–39/mo	\$14–30/mo	\$30–50+/mo
No CMS needed (we manage all updates)	Yes — files in repo	Still needs Webflow editor	Still needs Framer editor	WP admin still required
Platform lock-in risk	None — own the code	High	High	Medium
Update via Cursor / Claude Code	Native — it's just files	No	No	Partial
Existing stack alignment	Same as onboarding app	Separate platform	Separate platform	Separate platform
Maintenance overhead	Low	Medium	Medium	High (plugins, updates)

RISKS & MITIGATIONS

Known Risks

Template build is the critical path

No client sites can be delivered this way until the template is built. Continue WordPress for any immediate clients while template development runs in parallel.

MDX file conflicts on concurrent updates

If two pipeline jobs push to the same repo at the same time, a git conflict is possible. Mitigation: the push step uses GitHub's API with a read-then-write pattern and retries on conflict. In practice, CPA firm sites rarely see concurrent updates.

Next.js version maintenance

Next.js releases major versions roughly annually. Pin the version per client repo and upgrade deliberately. The template repo stays current; client repos are updated on a scheduled cadence.

Pipeline re-runs overwrite manual edits

If a page was hand-edited in Cursor after the initial publish, re-running the pipeline for that page would overwrite the edits.

Mitigation: track which pages have post-publish manual edits and warn before re-running. Or treat pipeline output as source of truth and keep all copy changes in the pipeline's schema.

NEXT STEPS

Recommended Sequence

- **Step 1:** Agree on the 12–15 blocks — define the full component inventory and which CSS custom properties each block exposes.
- **Step 2:** Build the Next.js template with all blocks and test rendering against the Korbey Lague content package.
- **Step 3:** Set up the Claude Design Project — load block docs, CSS variable reference, and design guidelines. Run a test session against a real client.
- **Step 4:** Define the MDX frontmatter schema and build the Pipeline → GitHub push step (content + theme.css).
- **Step 5:** Run the first full end-to-end flow: content pipeline → Claude Design session → Publish to Site → live Vercel deploy.
- **Step 6:** Document the per-client checklist and deliver the first client site on the new stack. Gather feedback before rolling out.

Summary

Build once, deploy many. The one-time template build enables low-cost, low-effort site delivery for every subsequent client.

No CMS required — CountingFive manages all content, so files in GitHub is the right model.

Per-site ongoing cost: ~\$11–11.50/month (Vercel \$10 + domain) vs. \$30–50+/month on WordPress.

All post-launch updates handled via the pipeline or directly in Cursor / Claude Code — no manual CMS login required.